Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 2

«Объектно-ориентированные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-31Б _____ (Арифулов Р.Х.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

1 Цель работы

Изучение объектно-ориентированных возможностей языка Python.

2 Задание

Вариант № 1 (N = 1).

– Создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием pip.

– Разработать программу, реализующую работу с классами, в виде консольного приложения на языке Python 3.

– Все файлы проекта (кроме основного файла main.py) должны располагаться в пакете lab_python_oop, каждый класс — в отдельном файле пакета.

– Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры.

– Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры.

– Класс «Прямоугольник» наследуется от класса «Геометрическая фигура» и содержит конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры». Класс переопределяет метод вычисления площади.

– Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус». Для вычисления площади используется константа math.pi.

– Класс «Квадрат» наследуется от класса «Прямоугольник» и содержит конструктор по длине стороны.

– Для классов «Прямоугольник», «Квадрат», «Круг» определить метод repr, который возвращает основные параметры фигуры, ее цвет и площадь (с использованием метода format). Название фигуры задается в виде поля данных класса и возвращается методом класса.

– В файле main.py создать прямоугольник синего цвета шириной N и высотой N, круг зеленого цвета радиусом N, квадрат красного цвета со стороной N и вывести о них информацию в консоль. Также вызвать один из методов внешнего пакета, установленного с помощью pip.

– Дополнительное задание: протестировать корректность работы программы с помощью модульного теста.

3 Текст программы

В качестве внешнего пакета используется пакет tabulate, с его помощью в конце программы выводится таблица с фигурами.

Файл lab_python_oop/figure.py:

```
from abc import ABC, abstractmethod


class Figure(ABC):

    @abstractmethod
    def square(self):
        pass
```

Файл lab_python_oop/color.py:

```
class FigureColor:

    def __init__(self):
        self._color = None

    @property
    def color(self):
        return self._color

    @color.setter
    def color(self, value):
        self._color = value
```

Файл lab_python_oop/rectangle.py:

```
from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor


class Rectangle(Figure):
    FIGURE_TYPE = 'Прямоугольник'

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, width, height, color):
        self.width = width
        self.height = height
        self.fc = FigureColor()
```

```
        self.fc.color = color

    def square(self):
        return self.width * self.height

    def __repr__(self):
        return '{} {} цвета шириной {} и высотой {}, площадь {}.'.format(
            self.get_figure_type(),
            self.fc.color,
            self.width,
            self.height,
            self.square()
        )
```

Файл lab_python_oop/circle.py:

```
import math

from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor


class Circle(Figure):
    FIGURE_TYPE = 'Круг'

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, radius, color):
        self.radius = radius
        self.fc = FigureColor()
        self.fc.color = color

    def square(self):
        return math.pi * self.radius ** 2

    def __repr__(self):
        return '{} {} цвета радиусом {}, площадь {:.3f}.'.format(
            self.get_figure_type(),
            self.fc.color,
            self.radius,
            self.square()
        )
```

Файл lab_python_oop/square.py:

```
from lab_python_oop.rectangle import Rectangle


class Square(Rectangle):
    FIGURE_TYPE = 'Квадрат'

    def __init__(self, side, color):
        self.side = side
        super().__init__(side, side, color)

    def __repr__(self):
        return '{} {} цвета со стороной {}, площадь {}.'.format(
            self.get_figure_type(),
            self.fc.color,
            self.side,
            self.square()
        )
```

Файл main.py:

```
from tabulate import tabulate

from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

N = 1


def main():
    r = Rectangle(N, N, 'синего')
    c = Circle(N, 'зеленого')
    s = Square(N, 'красного')
    print(r)
    print(c)
    print(s)

    print()
    table = [[f.get_figure_type(), f.fc.color, round(f.square(), 3)] for f in
(r, c, s)]
    print(tabulate(table, headers=['Фигура', 'Цвет', 'Площадь'],
tablefmt='grid'))


if __name__ == '__main__':
    main()
```

Файл test_figures.py:

```
import math
import unittest

from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square


class TestFigures(unittest.TestCase):

    def test_rectangle(self):
        r = Rectangle(2, 3, 'синего')
        self.assertEqual(r.square(), 6)
        self.assertEqual(r.fc.color, 'синего')

    def test_circle(self):
        c = Circle(2, 'зеленого')
        self.assertAlmostEqual(c.square(), 4 * math.pi)

    def test_square(self):
        s = Square(4, 'красного')
        self.assertEqual(s.square(), 16)
        self.assertEqual(s.get_figure_type(), 'Квадрат')
        self.assertIsInstance(s, Rectangle)


if __name__ == '__main__':
    unittest.main()
```

4 Экранные формы с примерами выполнения программы

Было создано виртуальное окружение venv и в него установлен пакет tabulate. Список пакетов виртуального окружения приведен на рисунке 1.

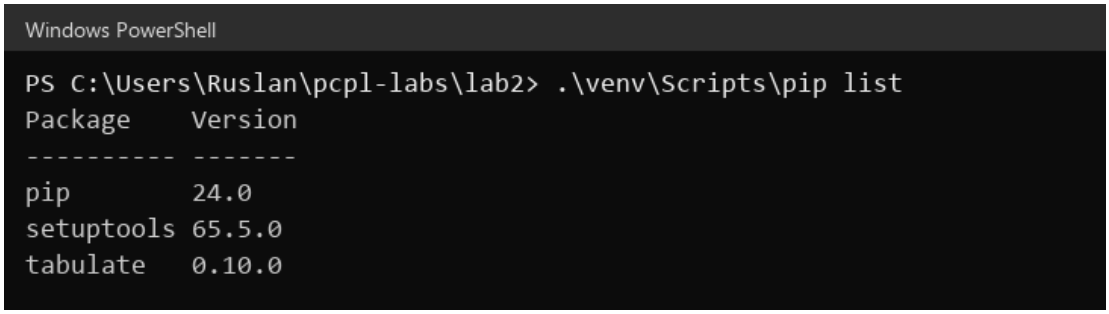


Рисунок 1 — Пакеты виртуального окружения

Результат выполнения программы main.py приведен на рисунке 2.

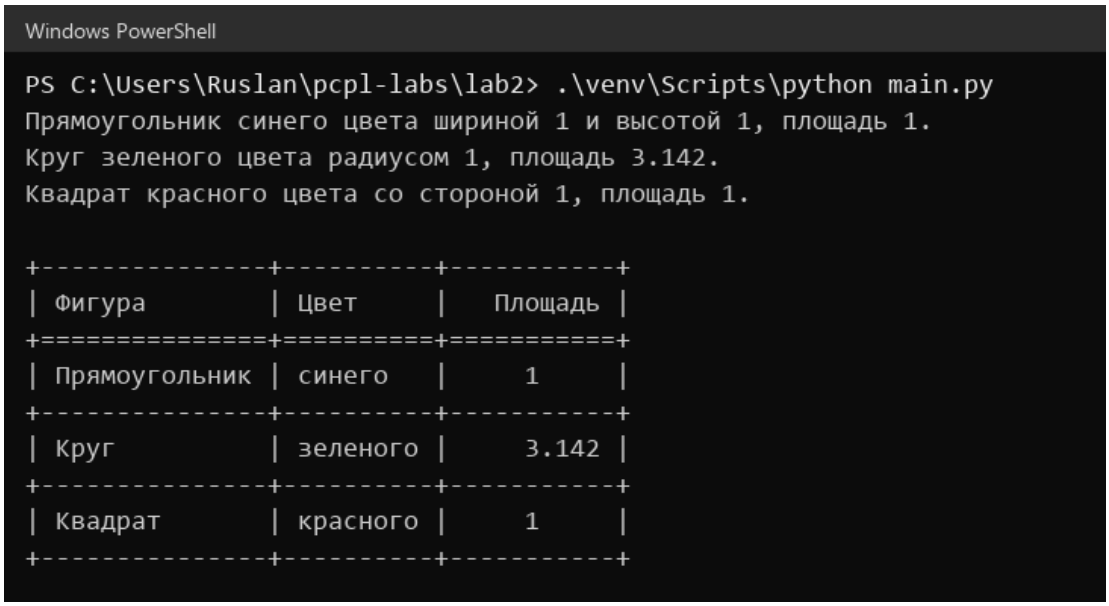


Рисунок 2 — Результат выполнения программы

Результат выполнения модульных тестов приведен на рисунке 3.

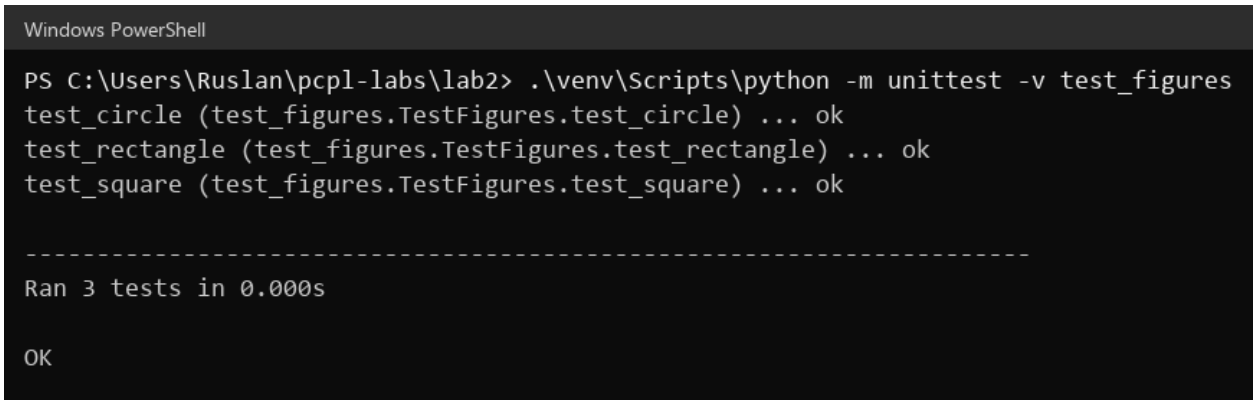


Рисунок 3 — Результат выполнения модульных тестов